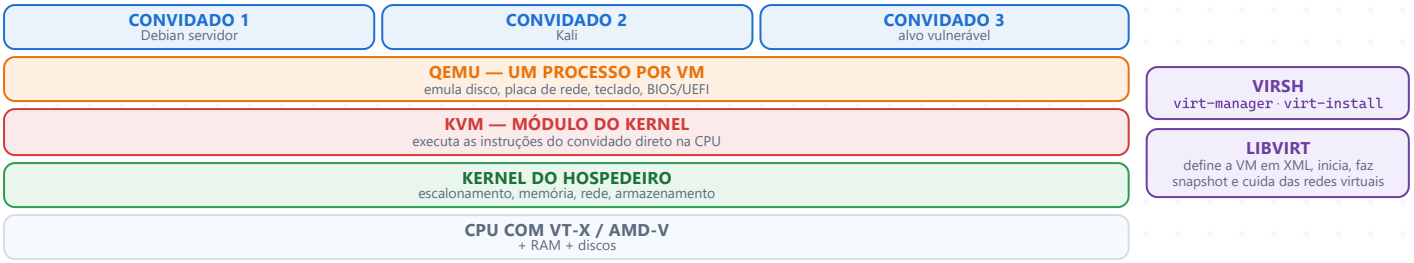
VIRTUALIZAÇÃO: KVM, QEMU E O LABORATÓRIO DE ESTUDOS

*Para praticar é preciso ter **onde errar**: um servidor para endurecer, um alvo para escanear, um sistema para quebrar e restaurar em trinta segundos. Nada disso cabe no notebook de trabalho, e alugar máquinas na nuvem para hospedar um alvo deliberadamente frágil é caro e irresponsável.*

TRÊS PEÇAS, TRÊS PAPÉIS — O QUE CADA CAMADA REALMENTE FAZ



O KVM não sabe o que é um disco — ele só dá acesso às extensões da CPU. Quem inventa os periféricos é o **QEMU**, delegando a execução ao KVM quando ele existe; sem KVM ele emula a CPU por software — **10 a 20× mais lento**, mas é assim que se roda ARM num x86. **O veredito é o `kvm-ok`:** deu vazio, o motivo mais comum não é o processador — é a extensão desligada no firmware UEFI, ou o seu Kali já ser uma VM sem *virtualização aninhada* (`/sys/module/kvm_intel/parameters/nested`). **E use `virtio` em disco e rede:** a otimização de maior retorno que existe.

O LABORATÓRIO: O MODO DE REDE SEPARA EXERCÍCIO DE RISCO



MODO	ALCANÇA · É ALCANÇADO
`isolated`	só os outros convidados · **ninguém de fora** — o modo dos **alvos vulneráveis**
`nat` (default)	internet e hospedeiro · só o hospedeiro — uso geral
`bridge`	a LAN inteira, com IP próprio · toda a LAN — só quando a VM tem mesmo de ser host da rede

É a ausência do elemento `<forward/>` no XML que torna a rede isolada: sem ele a libvirt não cria NAT nem rota — os convidados falam entre si e com o endereço da bridge, e nada mais. **A prova de que deu certo é um `ping` para a internet que *falha*** de dentro do convidado. Para instalar pacotes, troque a rede dele para `default` por tempo limitado e devolva à isolada **antes** de ligar o alvo frágil. (No QEMU puro há ainda o modo `user`: dá internet ao convidado por uma pilha em espaço de usuário, sem root e sem que ninguém o alcance — bom para teste rápido.)

NA PRÁTICA — DISCO DERIVADO, CLOUD-INIT E O SNAPSHOT QUE JUSTIFICA TUDO

kali@kali: ~

```
# disco derivado: parte da base, so grava a diferenca
$ sudo qemu-img create -f qcow2 -F qcow2 \
    -b /var/lib/libvirt/images/base-debian.qcow2 \
    /var/lib/libvirt/images/srv1.qcow2 20G
$ qemu-img info srv1.qcow2
virtual size: 20 GiB   disk size: 196 KiB
$ sudo virt-install --name srv1 --memory 1024 --vcpus 1 \
    --disk .../srv1.qcow2,bus=virtio --import \
    --os-variant debian12 --graphics none \
    --network network=lab-isolada,model=virtio
# o par que justifica o laboratorio inteiro:
$ virsh snapshot-create-as srv1 limpo "estado inicial"
$ virsh snapshot-revert srv1 limpo   # volta em segundos
$ virsh domifaddr srv1   # o IP que o convidado pegou
$ virsh console srv1     # serial; sair com Ctrl+]
```

O QUE USAR	POR QUÊ
`qcow2`	aloca sob demanda (40 GB declarados = 2 GB reais) e aceita snapshot no próprio arquivo. `vmdk` / `vdi`, só para importar VM pronta
`backing file`	dez VMs partem da mesma base somente-leitura e gravam só as diferenças — as camadas do contêiner, um nível abaixo
`nuvem + cloud-init`	dispensa instalar do zero: um `seed.iso` cria o usuário, injeta a chave SSH e instala pacotes no 1º boot
`snapshot ≠ backup`	depende do disco original e não sobrevive à falha do hospedeiro. Para preservar: `dumpxml` + cópia do `.qcow2` com a VM desligada

`virsh shutdown` **é ACPI e** `virsh destroy` **apenas arranca o cabo — quem apaga é o** `undefine`**, e** `virsh undefine srv1 --remove-all-storage` **apaga os discos sem pedir confirmação.** Se a VM usa disco derivado de uma base compartilhada, confira antes o que vai embora: `virsh domblklist <vm>` lista os arquivos. **Apagar a base leva junto todas as VMs derivadas dela** — e o erro só aparece na próxima vez que uma delas tentar ligar. Ensaie o ciclo criar/apagar com uma VM de teste antes de rodá-lo num laboratório que você não queira ter de remontar.

RESUMO RÁPIDO

- ✓ **VM traz o próprio kernel; contêiner compartilha o do hospedeiro.** Aplicação que você controla vai para contêiner; código não confiável, outro sistema operacional ou kernel que você *pretende* travar vai para VM.
- ✓ **Três peças: KVM** é o módulo que usa as extensões da CPU, **QEMU** é o processo que emula os periféricos, **libvirt** guarda a definição em XML e cuida de redes e snapshots.
- ✓ **Sem VT-x/AMD-V — ou sem virtualização aninhada, quando o hospedeiro já é VM — o QEMU emula por software** e fica 10 a 20× mais lento. `kvm-ok` dá o veredito; `virtio` dá o desempenho.
- ✓ `qcow2` **com disco derivado é o formato do laboratório**, e **imagem de nuvem + cloud-init** tornam a criação declarativa e repetível.
- ✓ **O modo de rede define o risco:** `isolated` para alvo vulnerável, `nat` para uso geral, `bridge` só quando a VM precisa ser host da LAN.
- ✓ **Snapshot é o que torna o laboratório utilizável** — quebrar e voltar em segundos. **Mas não é backup.** Uma VM que você tem medo de quebrar deixou de ser laboratório.

NO KALI

O laboratório de alvos vulneráveis é a razão nº 1 para virtualizar — e a rede isolada não é opcional. Metasploitable, DVWA e as máquinas do VulnHub têm falhas reais e sem correção. Em rede *bridge* ficam acessíveis a qualquer aparelho da sua LAN e, com encaminhamento de porta no roteador, à internet — e um alvo comprometido por terceiros para atacar outras pessoas a partir do seu IP é um **problema jurídico seu**. Regra fixa: **alvo vulnerável nasce e morre em rede** `isolated`. **Autorização vale dentro de casa:** praticar exploração é legítimo em máquinas que você criou ou em plataformas que autorizam por escrito (Hack The Box, TryHackMe, laboratórios de certificação). Escanear ou explorar sistema de terceiros sem autorização é crime (Lei 12.737/2012) — partir de uma VM não muda nada. **O laboratório existe para que não haja tentação de treinar em alvo real. Kali em VM é a instalação recomendada:** `kali-tweaks` cobre o que muda em convidado, e `spice-vdagent` (ou `open-vm-tools`) dá área de transferência e redimensionamento de tela. **Snapshot antes de cada sessão de estudo transforma um engano em trinta segundos de perda. Adaptador Wi-Fi USB precisa de passagem direta:** modo monitor não funciona pela placa emulada — repasse o dispositivo físico ao convidado (`virsh attach-device` com XML de `hostdev`), e o driver que precisa existir é o *do convidado*: DKMS outra vez, dentro da VM. **Analisar malware exige VM, não contêiner, com regras extras:** rede isolada ou desligada, snapshot antes e reversão depois, sem pasta compartilhada e sem USB. Amostras modernas detectam ambiente virtualizado e mudam de comportamento, e escape de hipervisor, embora raro, existe — para análise séria, hospedeiro dedicado e descartável. **Na RHEL:** `dnf group install "Virtualization Host"` e SELinux via `svirt` — mover um `.qcow2` na mão quebra o rótulo, e `restorecon -Rv` resolve.